

Assume:

- baseExpansionRadius = 3.0
- depthRadiusFactor = 0.65
- minExpansionRadius = 0.3

1. example tree

planet_0

├— sub_0 (has 2 children)

├— sub_1 (has 3 children)

├— sub_2 (has 4 children)

└— sub_3 (has 5 children)

2. The formula used

the readable radius is:

$$\text{effectiveRadius} = \max(\text{baseExpansionRadius} \times \text{depthRadiusFactor}^{\text{depth}}, \text{minExpansionRadius})$$

In our project:

- baseExpansionRadius = 3.0
- depthRadiusFactor = 0.65
- minExpansionRadius = 0.3

If the clicked parent is depth 0

$$\text{effectiveRadius} = \max(3.0 \times 0.65^0, 0.3) = \max(3.0 \times 1, 0.3) = 3.0$$

So for the **children of planet_0**, the radius is: **3.0**

3. actual calculation

Let us assume the 4 children of planet_0 have these CSV positions:

- sub_0 = (1, 1, 0)
- sub_1 = (2, 1, 0)
- sub_2 = (1, 3, 0)
- sub_3 = (4, 2, 0)

4. Step 1 — compute the centroid

The centroid means the **average center** of all children.

x-coordinate

$$(1 + 2 + 1 + 4)/4 = 8/4 = 2$$

y-coordinate

$$(1 + 1 + 3 + 2)/4 = 7/4 = 1.75$$

z-coordinate

$$(0 + 0 + 0 + 0)/4 = 0$$

So the centroid is:(2,1.75,0)

5. Step 2 — compute each child's offset from centroid

Now subtract the centroid from each child.

sub_0

$$(1,1,0) - (2,1.75,0) = (-1,-0.75,0)$$

sub_1

$$(2,1,0) - (2,1.75,0) = (0,-0.75,0)$$

sub_2

$$(1,3,0) - (2,1.75,0) = (-1,1.25,0)$$

sub_3

$$(4,2,0) - (2,1.75,0) = (2,0.25,0)$$

So the offsets are:

- sub_0 -> (-1, -0.75, 0)
- sub_1 -> (0, -0.75, 0)
- sub_2 -> (-1, 1.25, 0)
- sub_3 -> (2, 0.25, 0)

These offsets show the local shape of the children.

6. Step 3 — find how far each child is from the centroid

Now compute the length of each offset using distance formula:

$$\sqrt{x^2 + y^2 + z^2}$$

sub_0

$$\sqrt{(-1)^2 + (-0.75)^2} = \sqrt{1 + 0.5625} = \sqrt{1.5625} = 1.25$$

sub_1

$$\sqrt{0^2 + (-0.75)^2} = 0.75$$

sub_2

$$\sqrt{(-1)^2 + (1.25)^2} = \sqrt{1 + 1.5625} = \sqrt{2.5625} \approx 1.60$$

sub_3

$$\sqrt{2^2 + 0.25^2} = \sqrt{4 + 0.0625} = \sqrt{4.0625} \approx 2.02$$

So the distances are:

- sub_0 = 1.25
- sub_1 = 0.75
- sub_2 = 1.60
- sub_3 = 2.02

The farthest one is:

- $\text{sub_3} = 2.02$

So: $\text{maxDistance} = 2.02$

7. Step 4 — compute effective radius

Since `planet_0` is a top-level planet, its depth is 0.

So:

$$\text{effectiveRadius} = \max(3.0 \times 0.65^0, 0.3)$$

$$= \max(3.0, 0.3) = 3.0$$

So the farthest child should end up at distance **3.0** from the parent center.

8. Step 5 — compute the scaling factor

We now want to stretch the whole child pattern so that:

- old farthest child distance = 2.02
- new farthest child distance = 3.0

So the scale factor is:

$$\text{scaleFactor} = \frac{3.0}{2.02} \approx 1.485$$

This means we multiply **every child offset** by 1.485.

9. Step 6 — compute the new normalized offsets

sub_0

Old offset:

$$(-1, -0.75, 0)$$

New offset:

$$(-1 \times 1.485, -0.75 \times 1.485, 0) = (-1.485, -1.114, 0)$$

sub_1

Old offset:

$$(0, -0.75, 0)$$

New offset:

$$(0, -1.114, 0)$$

sub_2

Old offset:

$$(-1, 1.25, 0)$$

New offset:

$$(-1.485, 1.856, 0)$$

sub_3

Old offset:

$$(2, 0.25, 0)$$

New offset:

$$(2.97, 0.371, 0)$$

This is about 3 units away, which is correct.

10. Step 7 — place them around the parent's visible world position

Now suppose planet_0 is currently visible at: (10,5,2)

This is the parent's visible position in Unity.

Now add each normalized offset to this parent position.

sub_0 final position

$$(10,5,2) + (-1.485,-1.114,0) = (8.515,3.886,2)$$

sub_1 final position

$$(10,5,2) + (0,-1.114,0) = (10,3.886,2)$$

sub_2 final position

$$(10,5,2) + (-1.485,1.856,0) = (8.515,6.856,2)$$

sub_3 final position

$$(10,5,2) + (2.97,0.371,0) = (12.97,5.371,2)$$

So these 4 children are now spread into a readable arrangement around planet_0.

11. What exactly happened here?

We did **not** place them randomly.

We did **not** make them perfectly equally spaced.

We did:

- keep the child pattern shape
- keep left/right/up/down direction
- keep which one is closer and which one is farther
- stretch the pattern so the farthest one reaches radius 3.0

So the local child cluster became readable.

12. Now let us do the next level briefly

12. Now let us do the next level briefly

Suppose you click sub_1, and it has 3 children.

Now the same process happens **again**, but on those 3 children.

The only difference is:

- now the parent depth is deeper

Suppose sub_1 is at depth 1.

Then:

$$\text{effectiveRadius} = \max(3.0 \times 0.65^1, 0.3)$$

$$= \max(3.0 \times 0.65, 0.3) = \max(1.95, 0.3) = 1.95$$

So now the next set of children is spread into radius **1.95**, not 3.0.

That is how deeper levels become more compact.

NOTE:

Simple example

1. If we start with the 3 planets:

- planet_0
- planet_1
- planet_2

these are the top level.

That is usually called: **depth 0**

those are one level below the planet.

That means: **depth 1**

Now we click sub_1, and it shows more children:

- child_a
- child_b
- child_c

those are even deeper.

That means: **depth 2**

If we click again and go deeper:

- next level = **depth 3**

and so on.

```
planet_0      -> depth 0
├── sub_0      -> depth 1
│   ├── child_a -> depth 2
│   │   └── leaf -> depth 3
```

explanation is correct for the **node-child placement idea**, but the actual code also has a **single-child special case**:

- if a parent has only **one child**
- that child is placed directly at the parent's position

Updated single-child behavior

Single-child special case (current behavior):

If a parent has only one child, the child is **not** placed directly at the parent's position anymore.

Instead, the code now:

1. takes the **direction** from the parent's CSV position to the child's CSV position
2. normalizes that direction
3. places the child at:

$parentWorldPos + direction.normalized \times effectiveRadius$

If the direction is effectively zero, the code uses a safe fallback direction (`Vector3.up`) with the same `effectiveRadius`.

//

When planet_0 is clicked, and these 4 sub-planets are overlapping,
how does the formula separate them?

The answer is:

- we take the 4 child positions from the CSV
- compute their centroid
- compute offsets from centroid
- find the farthest one
- compute the effective radius
- scale all child offsets so the farthest one fits that radius
- then place that local layout around the parent's visible position

//